



HOCHSCHULE HEILBRONN

Labor Robotik

Seminararbeit SoSe 2025

„Integration eines Schunk Greifer EGK25 mit einem Universal Robot“



Bearbeiter: Roberto Nicklas

Matrikelnummer: 211963

Sommersemester 2025

Hochschule Heilbronn

Studiengang: Mechatronik und Robotik

Betreuer: Prof. Andreas Hoch, Georg Wörle

I. Inhaltsverzeichnis

I. Inhaltsverzeichnis	2
II. Abbildungsverzeichnis	4
III. Tabellenverzeichnis	4
1. Einleitung	5
2. Problemstellung	8
3. Stand der Technik	10
3.1 Robotik- und Greifergrundlagen	11
3.2 Kommunikationsprotokolle	12
3.3 URCap	13
3.4 Programmierung und Softwareumgebung	15
3.5 Firmware	15
4. Erster Lösungsansatz	17
4.1 URCap-basierte Integration als erster Ansatz	17
4.2 Bewertung der Umsetzbarkeit auf CB3	18
4.3 Ausblick auf einen zweiten Ansatz	18
5. Einrichtung der Entwicklungsumgebung	20
5.1 Vorbereitung	20
5.2 Implementierung, Testläufe, Build-Probleme	22
5.3 Beispiel für einen URCap Installationsnode in Java	23
5.4 Erkenntnisse	26
6. Durchführung mit Raspberry Pi	28
6.1 Aufbau und Kommunikation	28
6.2 Steuerung des Greifers über Modbus RTU mit Java	32
7. Ergebnisse und Bewertung	34
8. Ausblick	38
9. Fazit	40
10. Quellen- und Literaturverzeichnis	41
11. Anhang	42
A: ExampleInstallationNode.java	42
B: ExampleInstallationNodeView.java	42
C: build.gradle	43
D: SchunkGripperControl.java	43

E: Java-Beispielprogramm Modbus-RTU-Steuerung	44
---	----

II. Abbildungsverzeichnis

Abbildung 1 UR5 Roboter im Labor	5
Abbildung 2 Schunk Greifer EGK25	6
Abbildung 3 Fehlgeschlagener URCap Load.....	8
Abbildung 4 URCaps Auswahl am Teach Pendant	13
Abbildung 5 neuer Softwarestand.....	16
Abbildung 6 Screenshot Linux auf VM.....	21
Abbildung 7 Screenshot INTELLIJ.....	23
Abbildung 8 URCap Fehlermeldung	26
Abbildung 9 USB zu RS485 Adapter	29
Abbildung 10 USB Ports des Raspberry PI	29
Abbildung 11 Screenshot Projekt.....	30
Abbildung 12 Bootsequenz aus Anleitung	31
Abbildung 13 Terminalausgabe Raspberry PI mit Zyklus-Log	35
Abbildung 14 Status LED Greifer während Booten.....	37
Abbildung 15 Status LED Greifer nach Booten.....	37

III. Tabellenverzeichnis

Tabelle 1 RTU und TCP Merkmale	13
--------------------------------------	----

1. Einleitung

Die Robotik entwickelt sich seit Jahren zu einem wichtigen Bestandteil moderner Technik. Dabei kommt es insbesondere auf Präzision, Zuverlässigkeit und Flexibilität an. Die Leistungsfähigkeit eines Roboters hängt dabei maßgeblich von der eingesetzten Greifertechnik ab, da der Greifer den direkten Kontakt zum Werkstück herstellt und somit das gesamte Handling beeinflusst. An Hochschulen gewinnt die praktische Arbeit mit Robotern zunehmend an Bedeutung. Kollaborative Roboter (Cobots) von Universal Robots, werden bevorzugt eingesetzt, um Studierenden und Forschenden einen praxisnahen Zugang zu Aufbau, Programmierung und Erweiterung robotischer Systeme zu ermöglichen. Im Labor lassen sich Prozesse realitätsnah testen, ohne auf industrielle Produktionsanlagen angewiesen zu sein.

Für die praktische Umsetzung wird ein Universal Robots UR5 (Abbildung 1) mit dem Schunk Parallelgreifer EGK25-MB-M-B eingesetzt. Die serielle Modbus-RTU-Anbindung erfolgt über einen Raspberry Pi 4 als externes Gateway in Kombination mit einem USB-RS485-Adapter. Die Implementierung und Tests wurden unter Linux (VM) mit Java/Gradle durchgeführt; zum Abgleich dienten das Schunk Control Center und ein Telegramm-Analyzer.



Abbildung 1 UR5 Roboter im Labor

Kapitel 2 beschreibt die Problemstellung sowie die relevanten Randbedingungen der CB3-Plattform. Darauf aufbauend fasst Kapitel 3 die technischen Grundlagen zusammen, die für das Verständnis der Anbindung erforderlich sind, darunter Greiferprinzip und Registerrückmeldungen, Modbus-Kommunikation, URCap-Umfeld und die verwendete Toolchain. In den Kapiteln 4 und 5 werden der erste Lösungsansatz sowie die Einrichtung der Entwicklungs- und Testumgebung eingeordnet. Kapitel 6 dokumentiert anschließend die praktische Umsetzung über den Raspberry Pi als Gateway und die daraus abgeleiteten Kommunikationsabläufe. Abschließend werden in den Kapiteln 7 bis 9 die Ergebnisse bewertet und in einen Ausblick sowie ein Fazit überführt.

Der Greifer spielt in diesem Kontext eine zentrale Rolle. Er bestimmt, wie der Roboter mit seiner Umgebung interagiert, und beeinflusst Genauigkeit, Sicherheit und Effizienz der Anwendung. Neben einfachen mechanischen Greifern kommen zunehmend elektrische Varianten mit digitalen Schnittstellen zum Einsatz. Diese ermöglichen nicht nur präzise Bewegungen, sondern liefern zusätzlich Informationen über Position, Kraft und Betriebszustände. Ein Beispiel für einen solchen Greifer ist der bereits genannte Schunk EGK25-MB-M-B (siehe Abbildung 2). Die Integration eines solchen Greifers in bestehende Robotersysteme erfordert jedoch eine sorgfältige Planung, insbesondere wenn Komponenten unterschiedlicher Generationen zusammenarbeiten. Eine stabile und zuverlässige Kommunikation ist dabei entscheidend.



Abbildung 2 Schunk Greifer EGK25

In dieser Seminararbeit wird untersucht, wie der Schunk EGK25-Greifer in ein UR5-System der CB3.1-Generation integriert werden kann, um eine zuverlässige und nachvollziehbare Steuerung zu erreichen. Dabei zeigt sich früh, dass die CB3-Controller-Architektur moderne Greiferanbindungen, insbesondere serielle Modbus-Kommunikation, nur eingeschränkt unterstützt. Aus diesem Grund wird in der Arbeit ein weiterer Ansatz verfolgt, bei dem ein Raspberry Pi als Kommunikations-Gateway eingesetzt wird, um die Modbus-RTU-Kommunikation mit dem Greifer zu übernehmen. Der Fokus der Arbeit liegt auf dem Erkenntnisgewinn aus der praktischen Umsetzung im Labor und nicht auf industriellen Anwendungen. Analysiert werden die technischen Voraussetzungen, geeignete Kommunikationswege sowie die Umsetzung einer funktionsfähigen Steuerstruktur. Ziel ist es, eine Lösung zu entwickeln, die im Labor zuverlässig eingesetzt werden kann und sich für zukünftige Projekte leicht wiederverwenden lässt.

2. Problemstellung

Die Integration des EGK25-MB-M-B Greifers in die CB3-Roboterplattform ist technisch nicht direkt vorgesehen. Der Greifer wird über Modbus RTU oder Modbus TCP angesprochen und benötigt dafür eine zuverlässige Kommunikationsschnittstelle sowie eine Auswertung der Rückmelderegister. Die CB3-Generation von Universal Robots unterstützt solche modernen, Modbus-basierten Greifer nicht nativ. Zwar können über URCaps externe Geräte eingebunden werden, jedoch sind auf CB3-Controllern externe Bibliotheken, serielle Schnittstellen und bestimmte Kommunikationsressourcen eingeschränkt. Dadurch ist die direkte Umsetzung einer vollständigen Modbus-Kommunikation auf dem Robotercontroller nur begrenzt möglich. [1]

In der folgenden Abbildung 3 ist ein Screenshot des Versuchs zu sehen, ein URCap für den Schunk-Greifer auf dem UR5-Roboter zu installieren. Der Ladevorgang schlägt jedoch fehl.

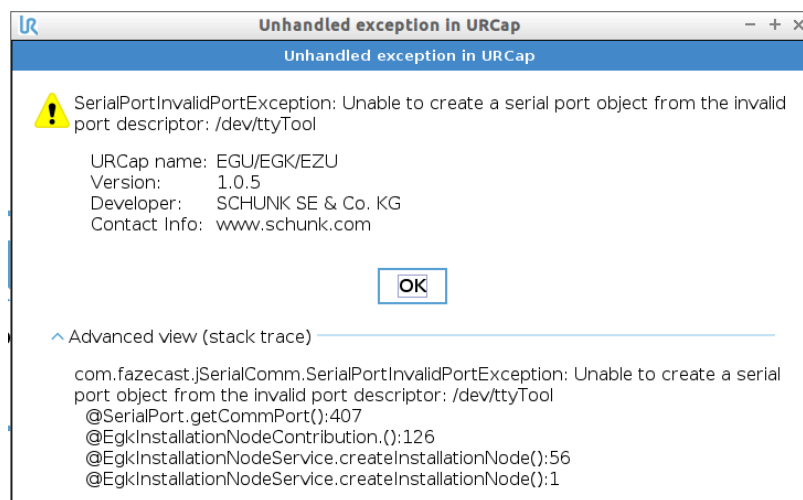


Abbildung 3 Fehlgeschlagener URCap Load

Der fehlgeschlagene Ladevorgang verdeutlicht, dass der URCap-basierte Integrationsweg in der Zielumgebung nicht zuverlässig nutzbar ist. Damit steht die zentrale Herausforderung im Vordergrund, wie die Modbus-Kommunikation und die erforderlichen Greifer-Rückmeldungen unter den CB3-Randbedingungen dennoch stabil umgesetzt werden können.

Ein reiner Auf-Zu-Betrieb über digitale Ein- und Ausgänge wäre prinzipiell möglich, bildet jedoch die Modbus-Funktionalität des Greifers nicht ab. Die verfügbare Rückmeldung über Register und Statusbits (z. B. Betriebszustand, Fehlerstatus, Positionswerte) würde dabei nicht genutzt und eine standardisierte Ansteuerung über Modbus entfele. Für die geplante Nutzung des EGK25 als Modbus-Greifer ist daher eine Lösung erforderlich, die die Kommunikations- und Rückmeldefunktionen integriert.

Vor diesem Hintergrund ergibt sich die zentrale Fragestellung der Arbeit: Wie kann ein moderner, Modbus-basierter Greifer in ein bestehendes Robotersteuerungssystem eingebunden werden, dessen Softwarearchitektur keine direkte Unterstützung für serielle Modbus-Kommunikation bietet, sodass Steuerung und Statusüberwachung zuverlässig und reproduzierbar erfolgen? Es gilt, die Anforderungen an Funktionsumfang und Standardisierung mit den Einschränkungen der CB3-Controller-Software in Einklang zu bringen, ohne dass bereits ein bestimmter Lösungsweg vorweggenommen wird.

3. Stand der Technik

Die aktuelle Forschung und industrielle Praxis zeigen, dass kollaborative Roboter in Kombination mit modularen Greifern an Bedeutung gewinnen. CB3-Roboter von Universal Robots, darunter UR3 und UR5, sind in vielen Anwendungen etabliert. Sie bieten eine modulare Softwarearchitektur, die Integration von URCaps und eine einfache Bedienung über das Teach-Pendant. Bewegungsprogramme lassen sich dort intuitiv erstellen und schnell an neue Aufgaben anpassen. [1]

Der EGK25-MB-M-B ist ein elektrischer Parallelgreifer mit definierten, konstanten Greifkräften. Zusätzlich liefert er über Modbus RTU oder Modbus TCP umfangreiche Statusinformationen. Dazu gehören Positionswerte, Greifkraftinformationen und codierte Fehler- bzw. Zustandsbits. In der Praxis ist die Integration solcher Greifer in Roboterumgebungen anspruchsvoll, da Bewegungsabläufe und Greiferaktionen eng abgestimmt werden müssen. Für industrielle Prozesse ist entscheidend, dass Befehle zuverlässig ausgeführt werden und Fehlerzustände eindeutig erkannt sowie korrekt behandelt werden können, z. B. durch Wiederholung eines Telegramms oder durch eine Rückmeldung an die Steuerung. [2]

Die Kommunikation zwischen Steuerung und Greifer erfolgt über Modbus, entweder seriell (RTU) oder über Ethernet (TCP). In dieser Arbeit liegt der Fokus auf Modbus RTU, da damit eine direkte serielle Verbindung zwischen Raspberry Pi und Greifer umgesetzt wird. [2] [3] [4] [5]

In der industriellen Anwendung wird Modbus RTU auf CB3-Systemen häufig nicht direkt im Robotercontroller umgesetzt. URCaps erlauben keinen nativen Zugriff auf serielle Schnittstellen, und zusätzliche Software-Layer können Timing und Timeouts beeinflussen. Daher werden in der Praxis oft externe Gateways eingesetzt. Ein separates System übernimmt die Modbus-Kommunikation, während der Robotercontroller nur einfache, klar definierte Befehle übergibt. Diese Trennung reduziert Komplexität im Robotercontroller und erhöht die Stabilität. [1]

3.1 Robotik- und Greifergrundlagen

Für das Projekt stehen die kollaborativen Roboter UR5 und UR3 der CB3-Reihe von Universal Robots zur Verfügung. Beide Systeme besitzen sechs Freiheitsgrade (6-DoF), ein Teach-Pendant zur Bedienung sowie das URCap-Konzept zur Erweiterung von Software- und Hardwarefunktionen. Auf CB3-Controllern bestehen jedoch Einschränkungen bei der Einbindung externer Bibliotheken und bei der Nutzung serieller Schnittstellen. Moderne URCap-Erweiterungen von Greiferherstellern, die eine direkte Plug-and-Play-Integration ermöglichen, sind daher auf CB3-Robotern in der Regel nicht kompatibel und werden primär für die e-Series bereitgestellt. [1]

Als Greifer wird der elektrische Parallelgreifer Schunk EGK25-MB-M-B eingesetzt. Die Ansteuerung erfolgt über Modbus RTU oder Modbus TCP. Neben der Fingerbewegung stellt der Greifer Status- und Prozessdaten bereit, darunter Positionswerte, Kraftwerte und Betriebszustände. Die Rückmeldungen sind über fest definierte Registeradressen und Statusbits abgebildet. Dadurch wird eine kontinuierliche Überwachung möglich, beispielsweise hinsichtlich Greiferbereitschaft, Bewegungszustand und Fehlerzuständen sowie Soll-/Ist-Werten. [2]

Im Vergleich zu einer reinen digitalen IO-Ansteuerung (Auf/Zu) erlaubt die elektrische Ausführung eine erweiterte Parametrierung und Auswertung. Greifkräfte können eingestellt, Bewegungen gezielter ausgeführt und Betriebsdaten während des Betriebs erfasst werden. Für den Laborbetrieb ist dies insbesondere relevant, da Versuchsabläufe reproduzierbar dokumentiert und Mess- bzw. Zustandsdaten systematisch ausgewertet werden können.

Die Kombination aus CB3-Robotern und dem EGK25-Greifer erfordert somit die Berücksichtigung der jeweiligen Hard- und Softwaregrenzen. Die CB3-Controller unterstützen Modbus nicht nativ, und die URCap-Architektur erlaubt keine direkten seriellen Zugriffe. Gleichzeitig ist für die gewünschte Funktionalität eine zuverlässige Übertragung und Auswertung der Greifer-Rückmeldungen erforderlich. Diese Rahmenbedingungen bilden die Grundlage für die spätere Auslegung der Kommunikations- und Steuerungsarchitektur. [1] [2]

3.2 Kommunikationsprotokolle

Die industrielle Kommunikation nutzt eine Vielzahl von Feldbussen, die sich in Übertragungsart, Geschwindigkeit, Echtzeitfähigkeit und Komplexität unterscheiden. Zu den verbreiteten Protokollen zählen EtherCAT, Profinet und Modbus. EtherCAT ist ein hochperformantes Echtzeitprotokoll und wird häufig in Motion-Control-Systemen eingesetzt. Profinet (PROFINET IO) ist ein Ethernet-basiertes Protokoll für Automatisierungsnetzwerke und erlaubt eine vergleichsweise einfache Integration in bestehende IT-Strukturen. [3]

Modbus ist ein älteres, aber weiterhin weit verbreitetes Kommunikationsprotokoll. Es existiert als serielle Variante Modbus RTU sowie als Ethernet-basierte Variante Modbus TCP. Modbus RTU wurde 1979 von Modicon entwickelt und arbeitet über serielle Schnittstellen wie RS-232 oder RS-485. Das Telegrammformat ist kompakt und wird über eine CRC-Prüfsumme abgesichert. Modbus RTU gilt als robust gegenüber elektromagnetischen Störungen und eignet sich für kurze bis mittlere Distanzen. Üblich ist eine Master-Slave-Struktur mit eindeutig adressierten Registern, über die Werte gelesen und geschrieben werden. Die Einfachheit erleichtert Implementierungen, begrenzt jedoch komplexe Topologien und Sicherheitsmechanismen. [3]

Modbus TCP nutzt Ethernet als physikalische Schicht und ersetzt die CRC-Prüfung durch den MBAP-Header (Modbus Application Protocol Header). Die Adressierung erfolgt über IP, wodurch mehrere Geräte über Standardnetzwerkinfrastruktur eingebunden werden können. Die logische Struktur bleibt weitgehend identisch zu RTU, allerdings ist TCP für größere Distanzen und umfangreichere Netzwerkstrukturen besser geeignet. Modbus TCP besitzt keine nativen Sicherheitsmechanismen wie Authentifizierung oder Verschlüsselung. In Labor- und Testumgebungen ist dies meist akzeptabel, in produktiven Industrieanlagen kann es jedoch relevant werden. [3]

Ein wesentlicher Unterschied liegt in der Kommunikationslogik: RTU arbeitet seriell mit Master-Slave-Prinzip, TCP IP-basiert im Client-Server-Modell. RTU eignet sich für kurze, deterministische Kommunikationszyklen, während TCP eine flexible Einbindung in Ethernet-Netzwerke ermöglicht. Beide Varianten verwenden dieselben Registeradressen und Funktionscodes. Dadurch kann ein Greifer wie der Schunk EGK25 grundsätzlich über RTU oder TCP angesteuert werden, abhängig von Hardware, Distanz und Netzwerkarchitektur. [3]

Um die Unterschiede zwischen Modbus RTU und Modbus TCP für die Steuerung des Schunk EGK25-Greifers zu verdeutlichen, wird die folgende Tabelle 1 präsentiert. Sie fasst die wichtigsten Eigenschaften beider Protokolle zusammen. [4]

Tabelle 1 RTU und TCP Merkmale

Merkmal	RTU	TCP
Übertragung	Seriell (RS-232/RS-485)	Ethernet (IP)
Datenrate	1,2 kBaud – 115,2 kBaud	10 Mbit/s – 1 Gbit/s oder höher
Adressierung	8-Bit Slave-Adresse	32-Bit IP-Adresse
Fehlererkennung	CRC-Checksumme	MBAP-Header
Integration	Einfach, kostengünstig	Flexibel, skalierbar

3.3 URCap

Ein URCap ist ein Software-Plugin für Universal Robots, das zusätzliche Funktionen oder externe Geräte direkt in die Benutzeroberfläche des Teach-Pendants integriert. Schunk stellt hierfür URCaps bereit, um Greif- und Spannsysteme sowie weitere Endeffektoren technisch und funktional an UR-Roboter anzubinden. Die folgende Abbildung zeigt exemplarisch die Darstellung von URCaps auf dem Teach Pendant eines UR5-Roboters.

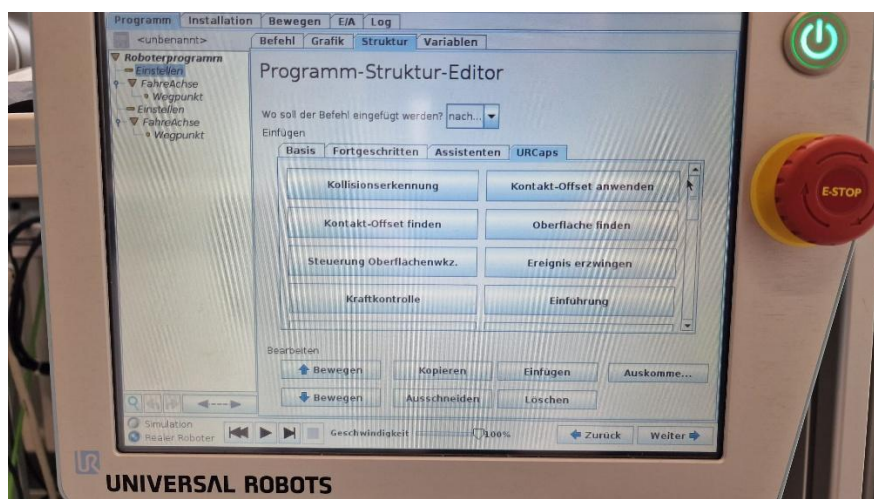


Abbildung 4 URCaps Auswahl am Teach Pendant

Ziel eines URCaps für den EGK25 ist eine möglichst direkte Einbindung des Greifers in die Robotersteuerung nach dem Plug-and-Play-Prinzip. Zentrale Parameter wie

Greifkraft, Hub, Geschwindigkeit und Betriebszustand werden dabei am Teach-Pendant angezeigt und können dort angepasst werden. Technisch besteht ein typisches URCap aus mehreren Bausteinen:

- Benutzeroberfläche (GUI): Bereitstellung von Menüs und Bedienseiten für Greiferfunktionen (z. B. Greifen, Loslassen) sowie Anzeige relevanter Zustände (z. B. „bereit“, „Fehler“, „Position erreicht“).
- Kommunikationsschnittstelle: Kopplung zwischen Plugin und Hardware. Je nach Greifer erfolgt dies über digitale I/Os, analoge Signale oder Feldbus-/Seriellprotokolle wie Modbus. Der Datenaustausch erfolgt typischerweise über die URCap-API und URScript.
- Treiber- und Logikmodule: Initialisierung und Parametrierung des Greifers (z. B. Kräfte, Geschwindigkeiten), zyklische Statusüberwachung und Fehlerbehandlung bei Kommunikationsproblemen oder ungültigen Zuständen.
- Integration und Parametrisierung: Bereitstellung von Standardwerten und Voreinstellungen zur schnellen Inbetriebnahme sowie Unterstützung typischer Integrationsanforderungen (z. B. mechanische Adapter, definierte Versorgung, Basis-Sicherheitsaspekte).

Schunk-Greifer sind elektromechanisch so ausgelegt, dass sie über standardisierte Flansche sowie Strom- und Signalversorgung am UR-Roboter montiert werden können. Über ein URCap lassen sich Hub, Greifkraft und gegebenenfalls Greifmodi einstellen. Die Software bildet dabei die internen Register und Statusbits ab und ermöglicht eine vollständige Steuerung, sofern die notwendige Kommunikationsanbindung und die Treiberunterstützung gegeben sind. [2]

Für den Einsatz eines Schunk URCaps müssen bestimmte Voraussetzungen erfüllt sein. Erforderlich sind eine passende Firmware- und Hardwareversion des Roboters, welche die URCap-API unterstützt, sowie eine korrekte elektrische Versorgung nach Spezifikation (typischerweise 24 V DC mit definierten Stromgrenzen und passenden Signalen). Zusätzlich müssen mechanische Kompatibilität und grundlegende Sicherheitsanforderungen berücksichtigt werden. Durch die Kombination aus grafischer Bedienoberfläche, Kommunikationsmodulen und integrierter Logik kann ein URCap die Inbetriebnahme beschleunigen, die Bedienung standardisieren und die Integration in Roboterprogramme vereinfachen. [1] [2]

3.4 Programmierung und Softwareumgebung

Für die Steuerung von Robotern und Greifern werden häufig Programmiersprachen eingesetzt, die stabil verfügbar sind und eine gute Unterstützung für Schnittstellen bieten. In diesem Projekt werden dafür Java und Python verwendet, da beide Sprachen in der Robotikpraxis verbreitet sind und sich funktional ergänzen.

Java ist insbesondere für die Entwicklung von URCaps relevant. Die objektorientierte Struktur unterstützt eine klare Trennung von Bedienoberfläche, Steuerlogik und Statusüberwachung. Dadurch lassen sich Funktionen nachvollziehbar strukturieren und in die Teach-Pendant-Oberfläche integrieren. Zusätzlich steht auf der UR-Controllerplattform eine Java-Laufzeitumgebung zur Verfügung, sodass URCap-Komponenten stabil betrieben werden können. [6]

Python wird vor allem für schnelle Implementierungen und die Anbindung externer Hardware eingesetzt. Durch die einfache Syntax und verfügbare Bibliotheken lassen sich Kommunikationsabläufe über serielle Schnittstellen oder Netzwerke zügig umsetzen und testen. Damit eignet sich Python insbesondere für Prototyping, Diagnose und die Kommunikation mit Geräten wie Greifern oder Sensoren. [6]

Beide Sprachen werden im Projekt zielgerichtet eingesetzt: Java deckt die Integration in die Roboterumgebung ab, während Python die flexible Anbindung eines externen Systems ermöglicht, beispielsweise eines Raspberry Pi als Gateway für die Greiferkommunikation. Dadurch entsteht eine Kombination aus stabiler Plattformintegration und anpassbarer Kommunikationslogik. [6]

In der vorliegenden Arbeit dienen Java und Python als Werkzeuge zur Umsetzung der Steuerungs- und Kommunikationsanforderungen. Die konkreten Bibliotheken, Schnittstellen und Implementierungsdetails werden in den nachfolgenden Kapiteln beschrieben.

3.5 Firmware

Die zwei Roboter UR3 und UR5 werden auf einen einheitlichen und aktuellen Firmwarestand der CB3-Reihe gebracht, um eine konsistente Systembasis für die weiteren Arbeiten sicherzustellen. Ein aktueller Firmwarestand ist erforderlich, damit Teach-Pendant, Controller und vorhandene Schnittstellen (z. B. Netzwerk, USB)

zuverlässig zusammenarbeiten und die vorgesehenen Integrationsschritte reproduzierbar durchgeführt werden können.

In der folgenden Abbildung (Abbildung 5.) ist der Versionsstand nach der Aktualisierung dokumentiert.



Abbildung 5 neuer Softwarestand

Ein Firmwarestand auf dem UR-Controller umfasst mehrere Komponenten, darunter Controller-Software, Teach-Pendant-Software und Kommunikationsanteile. Diese Komponenten müssen versionsseitig zueinander passen, da Abweichungen zu Instabilitäten führen können, beispielsweise bei der Bedienung, bei I/O-Funktionen oder bei der Nutzung von Schnittstellen für externe Geräte. Die Aktualisierung wird über das Teach-Pendant durchgeführt. Dazu wird das offizielle Firmware-Image auf einen USB-Datenträger übertragen und über das Einstellungs Menü eingespielt. Nach Abschluss des Update-Vorgangs startet das System neu. Anschließend erfolgt eine Funktionsprüfung, um sicherzustellen, dass Bedienung, Netzwerkverbindung und die relevanten Schnittstellen erwartungsgemäß arbeiten.

Damit bildet der dokumentierte Firmwarestand die Grundlage für die nachfolgenden Schritte der Systemintegration. Insbesondere werden dadurch Kompatibilitätsrisiken reduziert und die Voraussetzungen geschaffen, um weitere Softwarekomponenten und externe Kommunikationswege unter definierten Bedingungen zu testen. [1]

4. Erster Lösungsansatz

Ziel ist eine effiziente und stabile Steuerung des Schunk EGK25-MB-M-B innerhalb der gegebenen Systemgrenzen der CB3-Roboterplattform. Auf Basis der zuvor beschriebenen Grundlagen und Randbedingungen wird im Folgenden die erste Lösungsrichtung dargestellt, die sich aus der Projektplanung ergibt. Dabei steht zunächst die Frage im Vordergrund, wie sich eine möglichst direkte Bedienung und Rückmeldung des Greifers in der Roboterumgebung abbilden lässt, ohne zusätzliche Komponenten vorauszusetzen. Der Fokus liegt daher auf einem Ansatz, der die Greiferbedienung möglichst nah am Roboterprogramm hält und eine einheitliche Nutzung im Laborbetrieb ermöglicht. [1] [2]

4.1 URCap-basierte Integration als erster Ansatz

Der erste Lösungsansatz verfolgt eine möglichst zentrale Integration der Greiferfunktionalität über die Roboter Oberfläche. Damit sollen Greiferbefehle, Parametrierung und Statusanzeigen direkt dort verfügbar sein, wo auch die Bewegungsprogramme erstellt und ausgeführt werden. Aus Sicht der Projektplanung bietet dieser Weg den Vorteil, dass Bedienabläufe standardisiert werden können und die Greiferbedienung ohne zusätzliche externe Benutzeroberflächen oder separate Bedienrechner auskommt. [1]

In dieser Phase wird nicht strikt zwischen „Anpassung eines bestehenden URCaps“ und „Entwicklung eines eigenen URCaps“ unterschieden. Beide Wege werden als Varianten desselben Ansatzes verstanden, weil sie dieselbe Zielstruktur verfolgen: Die Greiferfunktionalität wird innerhalb der URCap-Architektur realisiert bzw. erweitert. Der Unterschied liegt lediglich darin, ob vorhandene Komponenten als Ausgangspunkt verwendet oder funktionsgleich neu aufgebaut werden. [1]

Für die Umsetzung werden im Rahmen der Planung drei Aufgabenbereiche als wesentlich identifiziert:

- **Bedien- und Parametrierebene:** Abbildung der relevanten Greiferfunktionen und Parameter in einer klaren Benutzerführung, einschließlich der Anzeige zentraler Zustände und Rückmeldungen.
- **Ablauf- und Statuslogik:** Implementierung einer definierten Initialisierung, zyklischer Statusauswertung und klarer Zustandsübergänge, sodass

Greiferaktionen reproduzierbar ausgeführt und Ausnahmesituationen nachvollziehbar behandelt werden können.

- **Kommunikationsanbindung:** Umsetzung der benötigten Kommunikationslogik innerhalb der URCap, inklusive der Abbildung der relevanten Register/Statusinformationen und einer konsistenten Fehlerbehandlung bei Übertragungs- oder Zustandsproblemen.

Damit beschreibt der erste Ansatz eine „integrierte Lösung“, bei der Greiferbedienung und Kommunikationslogik gemeinsam innerhalb der Roboterumgebung umgesetzt werden sollen. [1] [2] [3]

4.2 Bewertung der Umsetzbarkeit auf CB3

Die Planung zeigt, dass der URCap-basierte Ansatz grundsätzlich die gewünschten Vorteile für den Laborbetrieb bietet: eine direkte Bedienung, eine einheitliche Bedienlogik und die Möglichkeit, Greiferparameter sowie Rückmeldungen in unmittelbarem Zusammenhang mit dem Roboterprogramm zu verwenden. Damit wäre eine kompakte Systemarchitektur erreichbar, in der Bewegungsablauf und Greiferaktion in einer Umgebung zusammengeführt sind. [1]

Bei der Bewertung der Umsetzbarkeit treten jedoch die plattformbedingten Grenzen der CB3-Controller in den Vordergrund. Insbesondere betreffen diese Grenzen die Verfügbarkeit und Nutzbarkeit von Kommunikationsressourcen sowie die praktische Einbindung externer Komponenten innerhalb der Controllerumgebung. Dadurch wird die Umsetzung einer stabilen und vollständigen Kommunikationslogik innerhalb der URCap-Architektur in der Zielumgebung erschwert. Diese Einschränkungen wirken sich direkt auf die Zuverlässigkeit zyklischer Statusabfragen und auf die konsistente Ausführung komplexerer Befehlsfolgen aus. [1]

Aus diesem Grund wird der URCap-Ansatz im Projektverlauf als erster, naheliegender Weg verstanden, dessen Realisierbarkeit jedoch durch die CB3-Randbedingungen begrenzt ist. Die daraus resultierenden Risiken bilden die Ausgangsbasis für eine spätere Erweiterung der Betrachtung um alternative Architekturvarianten. [1]

4.3 Ausblick auf einen zweiten Ansatz

Im weiteren Projektverlauf wird deshalb zusätzlich ein zweiter Lösungsansatz betrachtet, bei dem die Kommunikationslogik außerhalb des Robotercontrollers

umgesetzt wird und der Robotercontroller nur klar definierte Befehle an eine externe Instanz übergibt. Dieser Ansatz wird in einem späteren Kapitel beschrieben, da er zum Zeitpunkt der initialen Planung noch nicht im Vordergrund steht und erst aus den praktischen Grenzen des ersten Ansatzes abgeleitet wird. [1]

5. Einrichtung der Entwicklungsumgebung

Für die Entwicklung und Testung der Softwarekomponenten wird eine stabile und reproduzierbare Entwicklungsumgebung benötigt. Ziel ist es, eine Plattform zu schaffen, auf der sowohl die Programmierung des URCap als auch die Kommunikation mit dem Schunk EGK25-Greifer technisch sicher und nachvollziehbar erfolgen kann. Da die offizielle Entwicklungsumgebung von Universal Robots auf Linux ausgelegt ist, bildet dieses Betriebssystem die Grundlage für den gesamten Softwareaufbau. Darüber hinaus soll die Umgebung flexibel genug sein, um spätere Erweiterungen wie Modbus TCP oder zusätzliche Peripheriegeräte ohne Neuaufsetzen des Systems zu ermöglichen.

Die folgenden Abschnitte beschreiben zunächst den Aufbau und die Begründung der gewählten Softwareumgebung und gehen anschließend auf die praktischen Schritte der Entwicklung ein, angefangen bei der Installation der Werkzeuge bis hin zu den ersten Versuchen und den aufgetretenen Problemen.

5.1 Vorbereitung

Für die Entwicklung von URCap und die Simulation eines UR5 CB3 ist eine stabile und reproduzierbare Systemumgebung essenziell. Universal Robots liefert das URCap SDK sowie URSim primär für Linux und testet dort die Schnittstellen. Aus diesem Grund wird Ubuntu LTS als Basis gewählt. Ubuntu bietet langfristige Sicherheitsupdates und eine breite Unterstützung für Entwicklerwerkzeuge. Die Nutzung von Linux reduziert Kompatibilitätsrisiken bei Java, bei Netzwerkdiensten und bei der Integration externer Bibliotheken.

Da die Arbeit auf einem Windowssystem stattfindet, wird Linux in einer virtuellen Maschine betrieben. Die Virtualisierung erlaubt es, die Linux-Umgebung zu snapshotten, zurückzusetzen und auf anderen Rechnern identisch bereitzustellen. Für die VM wird VMware Workstation oder VirtualBox eingesetzt, je nach Lizenzlage. Die VM bekommt dedizierte Ressourcen zugewiesen, typischerweise zwei virtuelle CPU-Kerne, vier Gigabyte RAM und 40 Gigabyte Festplattenspeicher, um URSim und den Buildprozess flüssig zu betreiben.

Die Hauptkomponenten der Umgebung sind:

- Ubuntu LTS als Betriebssystem

- Java Development Kit Version 17 als Laufzeitumgebung für URCaps
- URCap SDK in der jeweils empfohlenen Version laut Universal Robots Dokumentation

Dokumentation

- URSim zur Simulation des Roboters und der Panels
- Gradle als Build-System und optional Maven zur Interoperabilität
- Code-Editoren wie IntelliJ IDEA oder Visual Studio Code mit Java-Plugins
- Git für Versionsverwaltung
- Python 3 für Hilfsskripte und kleine Testtools

Die Installation erfolgt in klaren Schritten. Zuerst wird das System aktualisiert mit den Befehlen `sudo apt update` und `sudo apt upgrade`. Anschliessend wird das JDK installiert mit `sudo apt install openjdk-17-jdk`. Das URCap SDK wird von der Herstellerseite geladen, die Dokumentation wird lokal abgelegt und die Beispielprojekte werden in das Workspace Verzeichnis kopiert. URSim wird in einer separaten Partition oder einem separaten Verzeichnis installiert, die nötigen Umgebungsvariablen wie `JAVA_HOME` und `URCAP_HOME` werden permanent in der Shell-Konfiguration gesetzt. Abbildung 6 zeigt ein Beispiel wie ein Projektverzeichnis aussehen kann.

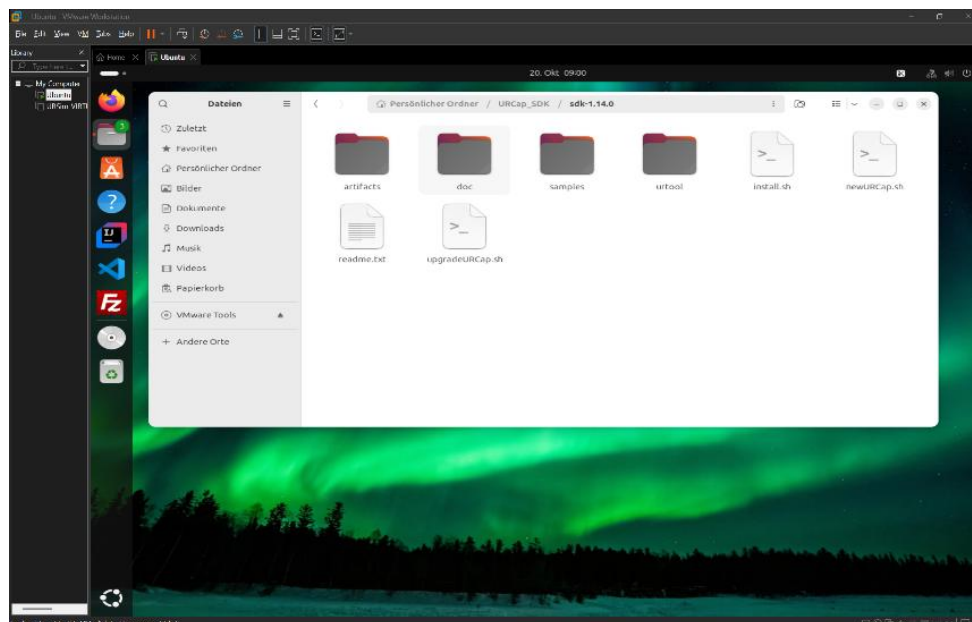


Abbildung 6 Screenshot Linux auf VM

Die Simulationsumgebung URSim stellt eine virtuelle Polyscope Oberfläche zur Verfügung, die genau das Verhalten des realen Controllers nachbildet. Gradle erleichtert die Verwaltung von Abhängigkeiten und das automatisierte Erzeugen der

distributeden URCap JAR Datei. Python wird bewusst optional gehalten, da es für schnelle Prototypen und für Analysezwecke sehr praktisch ist, aber die Produktionsimplementierung auf Java ausgelegt ist.

Die VM läuft in einem isolierten Netzwerkmodus, so dass URSim und Entwicklungsrechner per Loopback oder per Host-Only Interface kommunizieren, aber keine unnötigen externen Dienste exponiert werden. SSH Zugriff wird für Remote-Administration aktiviert. Die Benutzerkonten werden dokumentiert und ein Basisbackup der VM erstellt, bevor grössere Änderungen durchgeführt werden.

5.2 Implementierung, Testläufe, Build-Probleme

Nach der Vorbereitung beginnt die praktische Arbeit mit dem Erstellen erster URCap-Prototypen. Die Vorgehensweise ist iterativ. Zuerst entstehen minimale URCaps mit einfachen Panels, die jeweils nur einen Button und wenige Anzeigeelemente enthalten. Diese Minimalprojekte dienen dazu, den Buildprozess und die Integration in URSim zu prüfen. Ein typisches Testmuster ist: Panel erstellen, Gradle Build ausführen, URCap JAR in URSim importieren, Panel öffnen und Button klicken, Logs prüfen. Dieses Vorgehen schafft eine baseline für spätere Erweiterungen.

Sobald die Basis-Komponenten funktionieren, wird die Einbindung externer Bibliotheken geprüft. Für die Modbus-Kommunikation ist modbus4j eine naheliegende Bibliothek, weil sie Modbus RTU und Modbus TCP unterstützt. Bei der Einbindung zeigt sich jedoch ein Kernproblem. Modbus4j und einige andere benötigte Artefakte sind nicht über Maven Central verfügbar. Gradle kann deshalb die Abhängigkeiten nicht automatisch auflösen. Hier entstehen mehrere Handlungsoptionen, die jeweils praktischen Aufwand und Risiken mit sich bringen.

Option eins ist das manuelle Einbinden von JAR Dateien. Dazu werden die benötigten JARs in ein lib Verzeichnis im Projekt kopiert und in der Gradle Konfiguration als Dateien referenziert. Das ermöglicht lokale Kompilierung, kann aber zu Problemen führen, wenn URSim oder der CB3 Controller beim Laden des URCap die extrahierten Bibliotheken nicht akzeptiert. Solche Probleme treten genau dann auf, wenn das Zielsystem eine andere Classloader Struktur verwendet oder zusätzliche Security Policies greift.

Option zwei ist das Erzeugen eines lokalen Maven oder Gradle Repositories. Das sieht vor, die benötigten Artefakte in ein internes Repository zu deployen und Gradle darauf zu konfigurieren. Diese Lösung verbessert die Verwaltung, erhöht aber den Pflegeaufwand, weil jedes Update der Bibliothek manuell eingepflegt werden muss.

Option drei ist die Suche nach alternativen Bibliotheken oder dem Nachbau minimaler Modbus Funktionalität in reinem Java, um externe Abhängigkeiten zu vermeiden. Dies ist in vielen Fällen aufwändig, da Modbus detaillierte CRC Berechnungen, Byte Ordering und Funktionscodes verlangt. Ein selbst geschriebener Modbus Stack kann die Kontrolle erhöhen, kostet aber Zeit und birgt Fehlerpotenzial.

5.3 Beispiel für einen URCap Installationsnode in Java

Dieses Beispiel zeigt die Erstellung eines einfachen Installationsnodes für das URCap SDK in Java. Der Node besteht aus zwei wesentlichen Komponenten: der Installationsnode-Klasse `ExampleInstallationNode` und der zugehörigen View-Klasse `ExampleInstallationNodeView`.

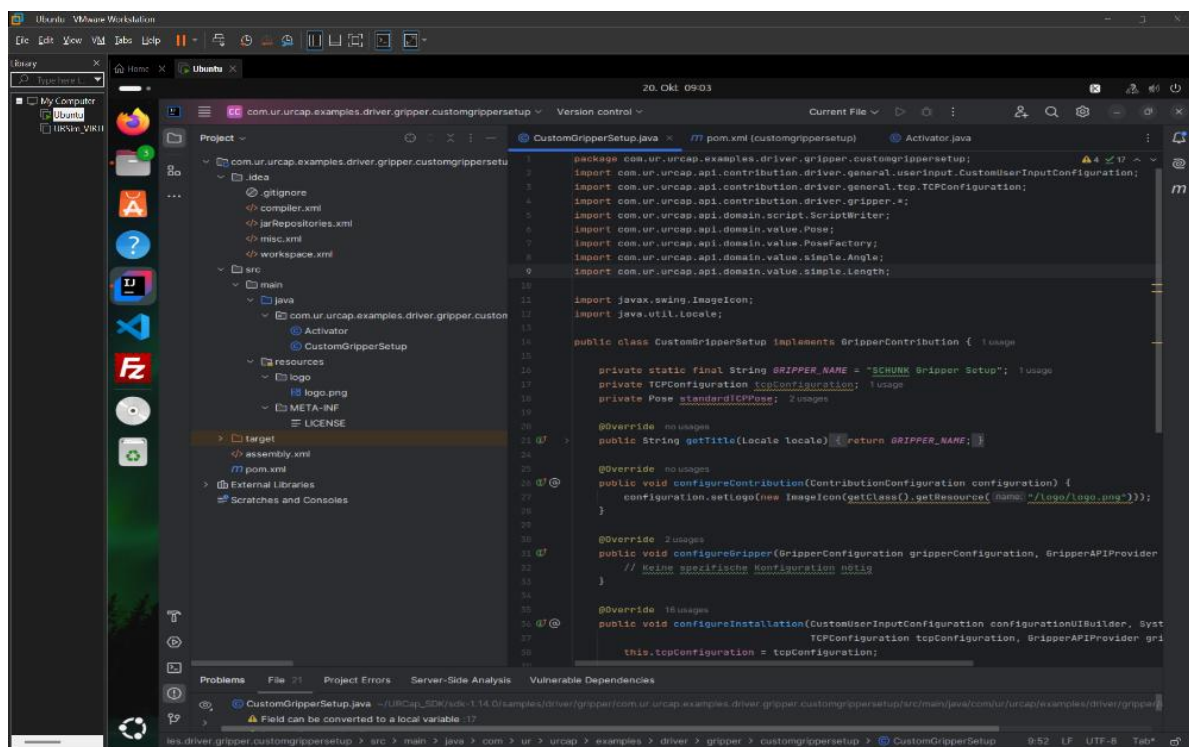


Abbildung 7 Screenshot INTELLIJ

Abbildung 7 zeigt das Interface von INTELLIJ für die Programmierung. Die vollständigen Quellcodes und die verwendete Build-Konfiguration sind im Anhang dokumentiert:

- Anhang A ExampleInstallationNode.java
- Anhang B ExampleInstallationNodeView.java
- Anhang C build.gradle (inkl. Kommentar zum Einbinden der URCap-SDK-JARs und dem Build-Befehl)

Die Klasse `ExampleInstallationNode` implementiert die Schnittstelle `InstallationNodeContribution` und steuert damit das Verhalten des Nodes im Teach Pendant. Beim Öffnen des Nodes ruft die Methode `openView()` die View auf und initialisiert die Benutzeroberfläche.

```
@Override
public void openView() {
    System.out.println("[URCap] Installation Node geöffnet - View wird geladen...");
    view.buildUI();
}
```

Die Methode `closeView()` wird beim Schließen des Nodes ausgeführt. Die Methode `generateScript()` erzeugt die URScript-Befehle, die später auf dem Roboter ausgeführt werden; im Beispiel wird lediglich ein Popup angezeigt.

```
@Override
public void generateScript(ScriptWriter writer) {
    writer.appendLine("popup(\"Hello from URCap Example on Ubuntu!\")");
}
```

Die Klasse `ExampleInstallationNodeView` erzeugt die grafische Oberfläche des Nodes. Die Methode `buildUI()` entfernt zunächst alle bestehenden Komponenten und fügt neue Elemente wie Labels und Buttons hinzu. Aktionen, wie das Anzeigen eines Dialogs beim Klicken eines Buttons, werden direkt registriert.

Mit der Methode **`getUI()`** liefert die View die Swing-Oberfläche an das URCap Framework zurück. Auf diese Weise trennt die Architektur die logische Steuerung des Nodes von der Benutzeroberfläche und ermöglicht eine modulare Struktur.


```
button.addActionListener(e -> JOptionPane.showMessageDialog(  
    null,  
    "Button gedrückt - funktioniert auch direkt auf dem Pendant!",  
    "URCap Test",  
    JOptionPane.INFORMATION_MESSAGE  
));
```

Die Kompilierung erfolgt über Gradle. In der build.gradle wird Java 17 als Kompilierungs- und Zielplattform definiert. Zusätzlich werden die URCap SDK JARs eingebunden, sodass alle URCap-spezifischen Klassen verfügbar sind. Mit dem Befehl ./gradlew clean build werden die Java-Dateien kompiliert und ein ausführbares JAR erzeugt. Dieses liegt nach dem Build im Ordner build/libs/ und kann direkt als .urcap auf dem Roboter installiert werden.

```
// build.gradle kompiliert unter Ubuntu 22.04 mit Java 17 und URCap SDK 1.15  
plugins {  
    id 'java'  
}  
  
// Pfad zum URCap SDK - muss an das lokale SDK-Verzeichnis angepasst werden  
dependencies {  
    implementation fileTree(dir: '/home/rob/urcaps/sdk/lib', include: '*.jar')  
}  
  
// Build-Einstellungen: Java 17, saubere Neu-Kompilierung und JAR-Erzeugung  
tasks.withType(JavaCompile) {  
    sourceCompatibility = '17'  
    targetCompatibility = '17'  
}  
  
// Erzeugt ein ausführbares .jar, das später als .urcap installiert werden kann  
jar {  
    archiveBaseName.set('ExampleURCap')  
    archiveVersion.set('1.0')  
}
```

Dieses ist die grundlegende Struktur eines Installationsnodes im URCap Framework und die Interaktion zwischen Node und View. Es bildet eine solide Grundlage für die Erweiterung um weitere Benutzeroberflächenkomponenten oder Roboterfunktionen.

5.4 Erkenntnisse

Die bisherigen Implementierungs- und Testläufe verdeutlichen, dass die Entwicklung eines funktionsfähigen URCaps auf der CB3-Plattform nicht nur durch Build- und Abhängigkeitsprobleme begrenzt ist, sondern vor allem durch systembedingte Einschränkungen der URCap-Sandbox. Abbildung 8 zeigt exemplarisch eine typische Fehlermeldung, die beim Laden des URCaps auf dem realen Controller auftritt, obwohl die Kompilierung lokal sowie der Test in URSim erfolgreich verlaufen sind. Diese Diskrepanz zwischen Entwicklungs- und Zielumgebung bildet die Grundlage für die folgenden Erkenntnisse und erklärt, warum bestimmte Ansätze trotz korrekter Implementierung nicht lauffähig sind.

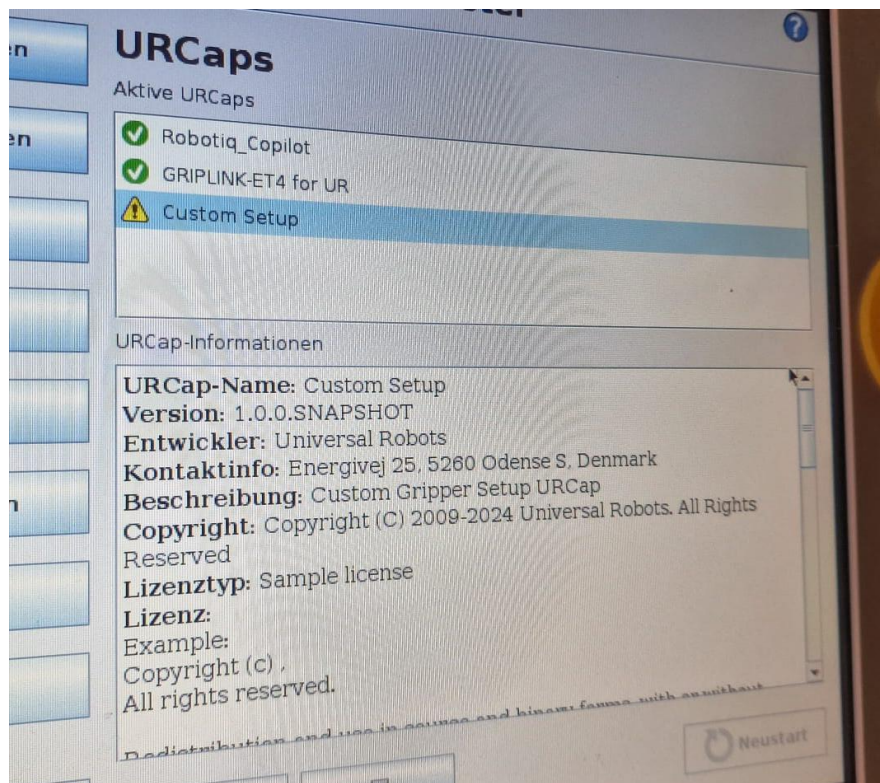


Abbildung 8 URCap Fehlermeldung

Parallel zu den Build-Herausforderungen treten Einschränkungen der Sandbox des URCap SDK auf. Die Sandbox erlaubt nicht den direkten Zugriff auf serielle Schnittstellen vom URCap aus. Das bedeutet, selbst wenn die Bibliotheken erfolgreich eingebunden sind, bleibt der Zugang zu /dev/ttyUSB0 oder zu RS485 Adaptern

blockiert. Diese Beschränkung ist designbedingt, um den Controller zu schützen. Aus Sicht der Systemarchitektur macht diese Restriktion Sinn, sie verhindert aber das zentrale Ziel, Modbus RTU direkt vom URCap aus zu betreiben.

Praktisch zeigen sich somit folgende Symptome. Lokale Kompilierungen laufen ohne Fehler, das URCap lässt sich in URSim laden, aber beim Versuch, Funktionen aus externen JARs zu nutzen, treten Laufzeitfehler auf oder das URCap verweigert die Initialisierung im echten Controller. In einigen Fällen erscheinen in den Logs Exceptions wie `ClassNotFoundException` oder `SecurityException`. Diese Fehler sind schwer zu beheben, weil sie aus dem Zusammenspiel von Classloader, Sandbox und Controllerlogik resultieren.

6. Durchführung mit Raspberry Pi

Auf Basis der bisherigen Erkenntnisse wird eine architekturelle Entscheidung getroffen. Der direkte URCap-Ansatz mit Modbus RTU auf dem CB3-Controller ist für das vorliegende Projekt nicht praktikabel. Stattdessen wird ein externer Kommunikationsgateway auf Basis eines Raspberry Pi 4 eingerichtet. Die Architektur besteht aus zwei getrennten Komponenten. Der Raspberry Pi übernimmt die vollständige Modbus-RTU-Kommunikation, während das URCap auf eine minimalistische Bedienoberfläche reduziert wird, die über eine Netzwerkverbindung einfache Befehle an den Pi übergibt. Diese Aufgabenteilung hat mehrere technische und organisatorische Vorteile.

6.1 Aufbau und Kommunikation

Der Raspberry Pi 4 wird als Kommunikations-Gateway eingesetzt, da er den direkten Zugriff auf serielle Schnittstellen und Systemfunktionen ohne die Einschränkungen des CB3-Controllers ermöglicht. Der Schunk EGK25-MB-M-B wird über einen USB-RS485-Adapter mit dem Raspberry Pi verbunden. Das serielle Interface erscheint unter Linux als Gerätedatei (z. B. `/dev/ttyACM0` oder `/dev/ttyUSB0`, abhängig vom Adapter). Für eine stabile Modbus-RTU-Kommunikation ist die korrekte Parametrierung der Schnittstelle erforderlich (Baudrate, Parität, Stopbits und Slave-Adresse). Bereits kleine Abweichungen führen typischerweise zu fehlerhaften Antworten oder Kommunikationsabbrüchen. Auf dem Raspberry Pi lassen sich zudem Telegrammaufbau, CRC-Prüfung, Timeouts und Wiederholmechanismen vollständig protokollieren und gezielt variieren, was eine systematische Fehlersuche und Stabilisierung ermöglicht. [3] [5]

Die Einrichtung des Systems beginnt mit der Installation von Raspberry Pi OS und der Einrichtung einer isolierten Entwicklungsumgebung. Für erste Funktionstests wird Python in einer virtuellen Umgebung verwendet (z. B. mit `pymodbus`). Parallel wird für die spätere Umsetzung und Stabilisierung der Kommunikationslogik eine Java-Umgebung mit OpenJDK 17 eingerichtet. Zur Verbindung wird ein USB-RS485-Adapter eingesetzt (siehe Abbildung 9), der vom Linux-System zuverlässig erkannt wird. [5]



Abbildung 9 USB zu RS485 Adapter

Die elektrische Verbindung zwischen Raspberry Pi und Greifer erfolgt gemäß RS485-Spezifikation. In Abbildung 10 sind die möglichen USB Ports für den RS485 Anschluss erkennbar. Zwei dieser Ports werden für Maus und Tastatur gebraucht. Die 24-V-Versorgung des Greifers wird über ein Labornetzgerät bereitgestellt. Bei der Verdrahtung wird darauf geachtet, Störeinflüsse und potenzielle Masseschleifen zu vermeiden, da diese die serielle Kommunikation beeinträchtigen können.

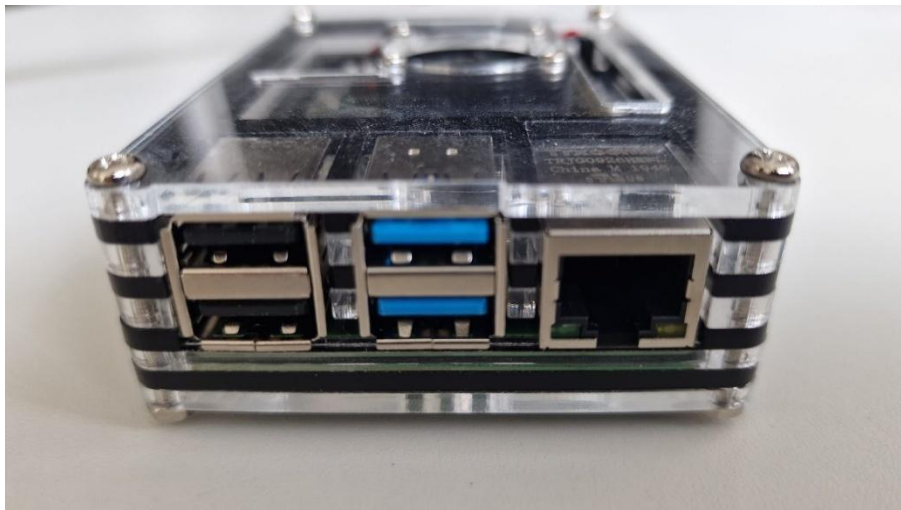


Abbildung 10 USB Ports des Raspberry Pi

Die Umsetzung erfolgt schrittweise: Zunächst wird die Registerkommunikation verifiziert, indem Status- und Identifikationsregister gelesen und die Antworttelegramme ausgewertet werden. Diese Phase dient dazu, Parameter (z. B. Baudrate/Parität) und die grundlegende Stabilität der Verbindung abzusichern. In der praktischen Erprobung zeigt sich, dass sich Python-Prototypen sehr gut für schnelle Verifikation und Analyse eignen, insbesondere für Lesezugriffe und für das Verständnis der Rückmelderegister. Für eine robuste, wiederholbare Steuerung inklusive stabiler Schreibzugriffe und definierter Sequenzen (Initialisierung, Freigabe,

Bewegungsbefehle) wird die Implementierung anschließend in Java überführt, da dort Timing, Pufferung und zyklische Abläufe gezielter kontrolliert werden können.

Auf Basis der gewonnenen Erkenntnisse wird die Kommunikationslogik in Java strukturiert umgesetzt. Die Implementierung umfasst dabei die Initialisierung des Greifers, das Quittieren des Default-Fehlerzustands, die Überwachung relevanter Statusbits sowie das Lesen und Schreiben von Holding-Registern. Ein wesentlicher Bestandteil ist die saubere Behandlung von Timeouts, Wiederholungen und die konsistente Auswertung der Greiferzustände, um einen reproduzierbaren Ablauf zu gewährleisten. Dadurch entsteht eine modulare Struktur, die sowohl für Testläufe als auch für spätere Erweiterungen geeignet ist. In folgender Abbildung 11 ist ein Screenshot vom verwendeten Projektverzeichnis auf dem Raspberry PI.

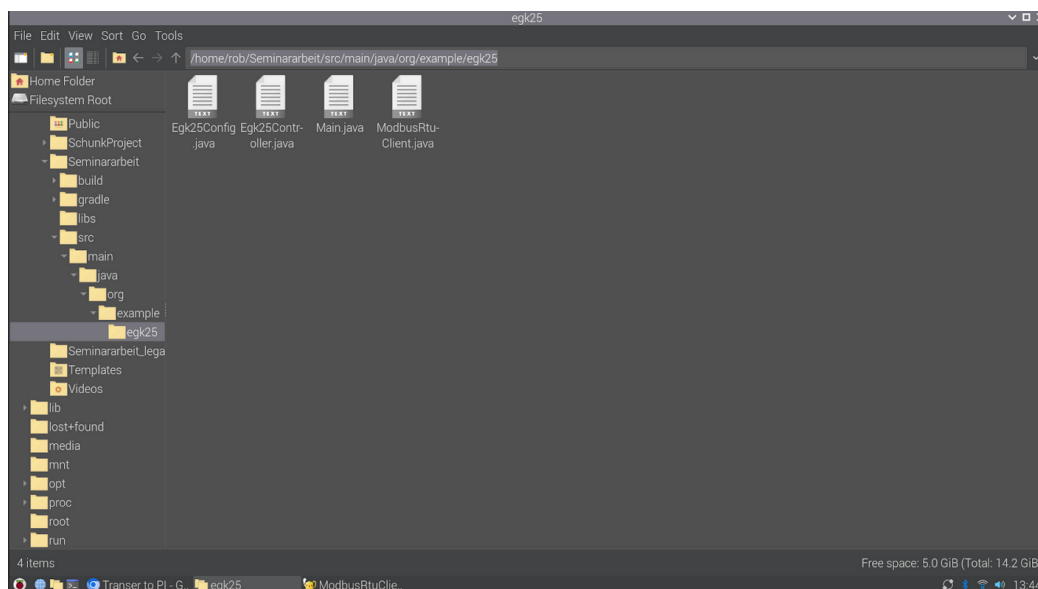
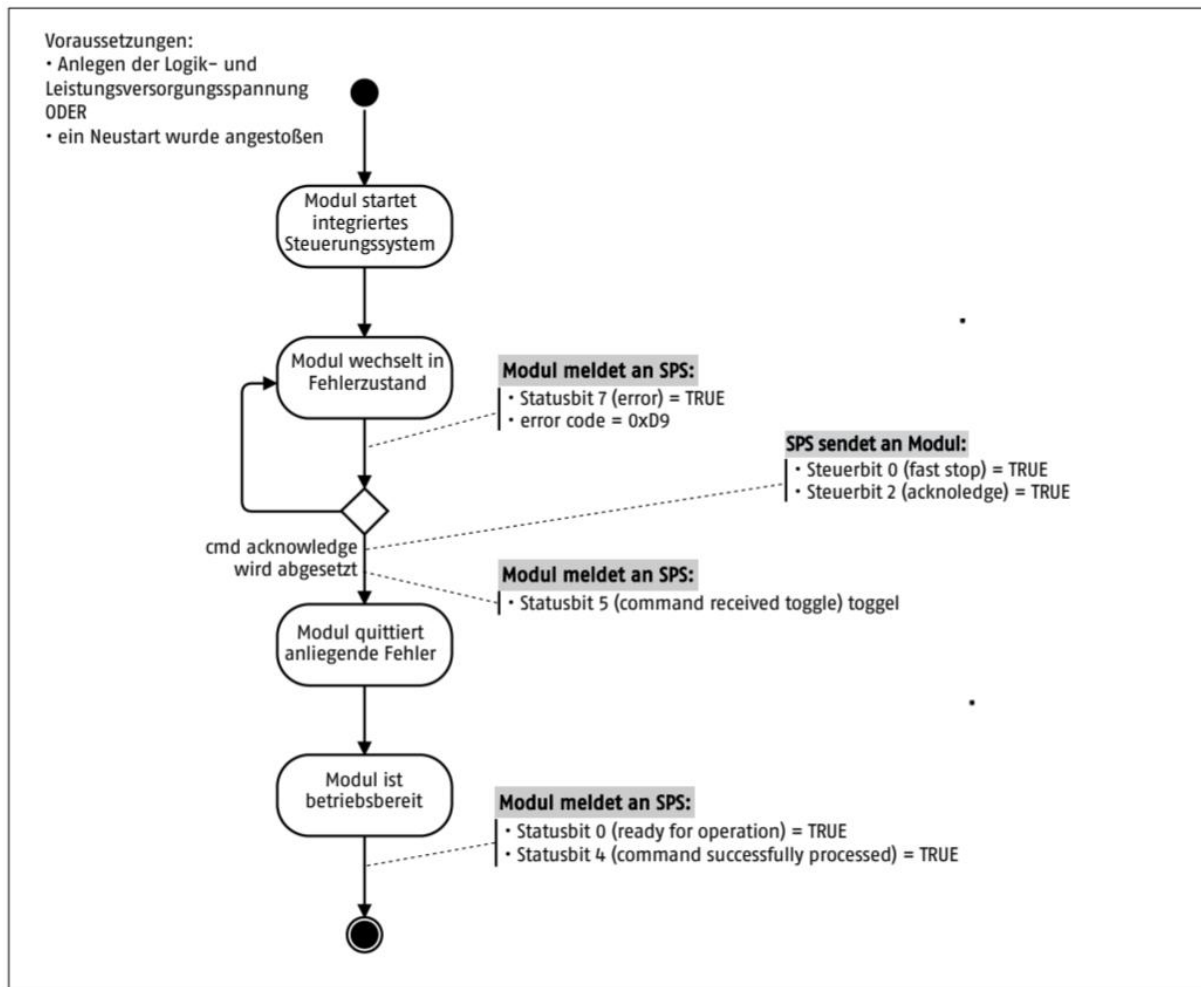


Abbildung 11 Screenshot Projekt

Die Abfolge der Initialisierung orientiert sich an der vom Hersteller beschriebenen Startsequenz. Abbildung 12 stellt den internen Ablauf von der Aktivierung bis zum betriebsbereiten Zustand schematisch dar. Daraus ergibt sich, dass bestimmte Freigabe- und Statusbedingungen (z. B. Enable/Valid und Ready-Status gemäß Mapping) in einer definierten Reihenfolge erfüllt sein müssen, bevor Bewegungsbefehle zuverlässig angenommen werden. Diese Abhängigkeiten werden in der Implementierung durch eine sequenzielle Befehlslogik abgebildet. [2]



Modul booten und Betriebsbereitschaft herstellen

Abbildung 12 Bootsequenz aus Anleitung

Parallel zur Implementierung wird die Kommunikation mit einem Telegramm-Analyzer (Free Device Monitoring) beobachtet. Damit lassen sich Modbus-RTU-Telegramme in Echtzeit aufzeichnen und hinsichtlich Struktur, Funktionscodes und Datenfeldern prüfen. Durch den Vergleich mit Referenztelegrammen aus dem Schunk Control Center werden Registeradressen, Datenbereiche und die erforderliche Befehlsreihenfolge validiert.

Insgesamt zeigt sich im Versuchsaufbau, dass eine zuverlässige Kommunikation vor allem von drei Faktoren abhängt: korrekte serielle Parametrierung, eine konsistente und vollständige Telegrammstruktur (inklusive CRC) sowie eine definierte Sequenz mit passenden Zeitabständen zwischen Schreib- und Lesezugriffen. Mit der Umsetzung

auf dem Raspberry Pi als Gateway können diese Faktoren gezielt kontrolliert und stabil realisiert werden, wodurch eine belastbare Basis für die Greifersteuerung entsteht.

6.2 Steuerung des Greifers über Modbus RTU mit Java

Im folgenden Abschnitt wird die Kommunikationslogik zwischen dem Raspberry Pi 4 und dem Schunk EGK25-MB-M-B Greifer beschrieben. Ziel der Implementierung ist der Aufbau einer stabilen seriellen Verbindung über Modbus RTU sowie die strukturierte Umsetzung zentraler Steuerfunktionen in Java. Dazu gehören die Initialisierung des Greifers, das Quittieren von Fehlerzuständen, die Aktivierung des Systems und die zyklische Abfrage relevanter Statusinformationen. Die gezeigten Codebeispiele dienen der nachvollziehbaren Darstellung der Kommunikationsabläufe und bilden die Grundlage für eine reproduzierbare und erweiterbare Greifersteuerung. [6]

Die Kommunikation erfolgt im Versuchsaufbau über den seriellen Port des RS-485-Adapters (z. B. /dev/ttyACM0). Der Raspberry Pi agiert als Modbus-Master, der Greifer als Slave (Unit-ID 10). Die Übertragung wird mit 8 Datenbits, gerader Parität und einer Baudrate von 19200 Bit/s betrieben. Der Code erzeugt die benötigten Modbus-RTU-Telegramme, berechnet die CRC-Prüfsumme und unterstützt sowohl Lese- als auch Schreibzugriffe über Modbus-Funktion 03 (Read Holding Registers) und Funktion 16 (Write Multiple Registers). [2] [3] [5]

Zur Überwachung der Verbindung wird zusätzlich eine zyklische Abfrage (Heartbeat/Keep-alive) implementiert. Dabei werden in regelmäßigen Abständen definierte Register gelesen, um Kommunikationsabbrüche frühzeitig zu erkennen und den aktuellen Gerätezustand kontinuierlich auszuwerten. Die Aktivität ist am Gerät typischerweise durch Kommunikationsindikatoren (z. B. COM-LED) erkennbar, ersetzt jedoch keine logische Statusprüfung im Programm. [3]

Während der Initialisierungsphase wird ein definierter Ablauf umgesetzt: Zunächst werden mögliche Fehlerzustände quittiert, anschließend wird der Greifer aktiviert (z. B. Enable/Valid gemäß Registermapping), und danach wird die Betriebsbereitschaft über Statusbits bzw. Registerwerte geprüft. Erst wenn der Greifer als bereit gemeldet wird und keine Fehlerzustände anliegen, werden weitere Steuerbefehle freigegeben. [2]

Der folgende Code zeigt eine Beispielimplementierung, die im Rahmen des Projekts auf dem Raspberry Pi 4 für Test- und Validierungszwecke ausgeführt wird:

1. Initialisierung des Greifers

Dieses Snippet zeigt, wie das Programm den Greifer vorbereitet. Es wird ein spezielles Register beschrieben, das den Greifer aus dem Default-Fehlerzustand in einen bereitbaren Zustand versetzt.

```
// Sende Initialisierungssignal an den Greifer  
writeRegister(port, START_REGISTER, INIT_SIGNAL);
```

2. Fehlerquittierung

Hier quittiert das Programm mögliche Fehler im Greifer, sodass er für weitere Befehle freigegeben wird. Die Schleife stellt sicher, dass alle fehlerhaften Zustände korrekt zurückgesetzt werden.

```
// ACK-Signal mehrfach senden, um Fehlerzustände zu löschen  
for (int i = 0; i < 4; i++) {  
    writeRegister(port, START_REGISTER, ACK_SIGNAL);  
    Thread.sleep(500);  
}
```

3. Aktivierung (Enable + Valid)

Dieser Befehl setzt die Steuerbits „Enable“ und „Valid“. Nur danach reagiert der Greifer auf Bewegungsbefehle. Ohne diesen Schritt bleibt der Greifer inaktiv.

```
// Greifer aktivieren  
writeRegister(port, START_REGISTER, ENABLE_VALID);
```

4. Statusabfrage

Das Programm prüft, ob der Greifer bereit ist und keine Fehler vorliegen. Der Status kann zusätzlich genutzt werden, um Bewegungen oder Warnungen auszulösen.

```
// Statusregister auslesen  
  
int status = readRegister(port, STATUS_REGISTER);  
  
System.out.println("Status: 0x" + Integer.toHexString(status));
```

Abschließend werden Statusregister gelesen und ausgewertet, um Betriebsbereitschaft und Fehlerfreiheit zu prüfen. Diese Informationen können als Freigabekriterium für nachfolgende Aktionen verwendet werden.

Die dargestellten Codeausschnitte fassen die Kernfunktionen der Java-Implementierung zusammen. Sie zeigen den strukturierten Ablauf aus Initialisierung, Fehlerbehandlung, Aktivierung und Statusprüfung, ohne den vollständigen Quellcode im Hauptteil abzubilden. Dadurch bleibt der Ablauf nachvollziehbar, während Detailimplementierungen (z. B. CRC-Berechnung, Telegrammaufbau, Hilfsmethoden) getrennt dokumentiert werden. [3]

Zur vollständigen Nachvollziehbarkeit ist die Java-Implementierung im Anhang in größerem Umfang aufgeführt. Dort sind die relevanten Klassen und Methoden dokumentiert, sodass die Kommunikationslogik reproduzierbar nachvollzogen und bei Bedarf erweitert werden kann.

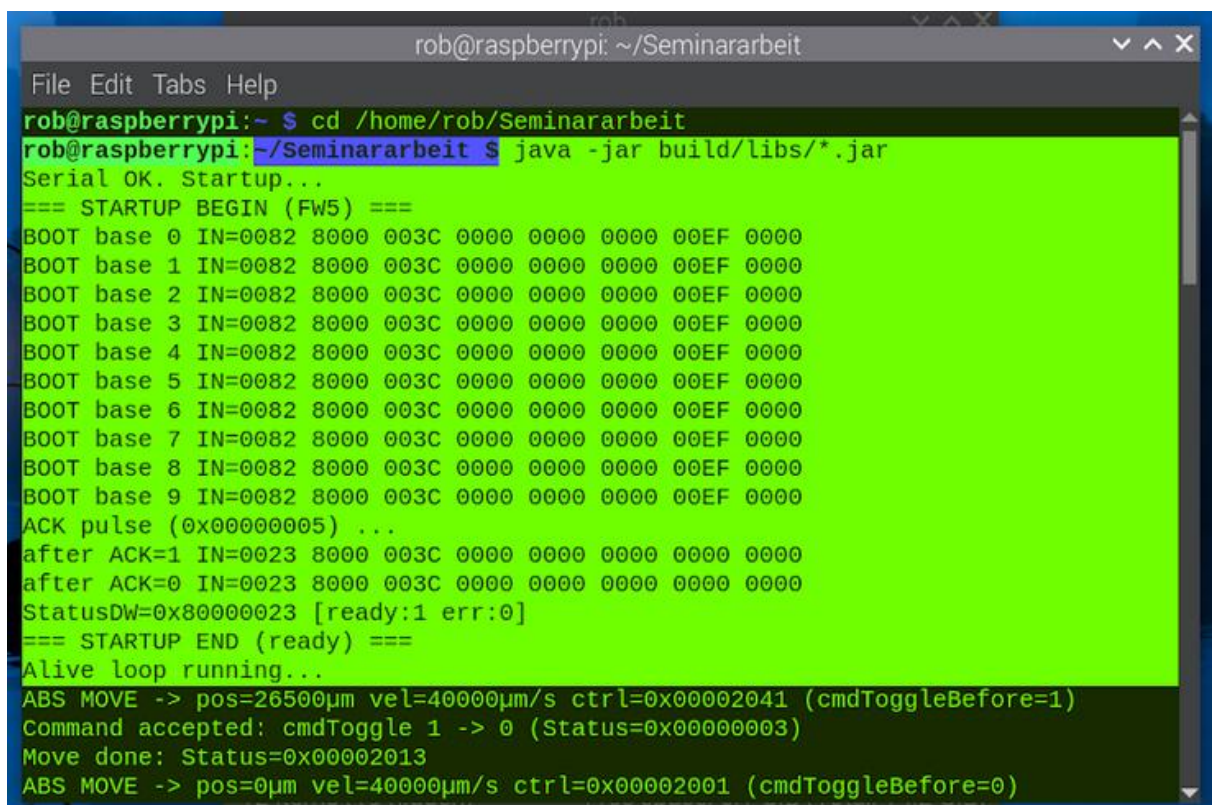
7. Ergebnisse und Bewertung

Im Verlauf der Arbeit wird ein strukturiertes Kommunikationskonzept zwischen dem Schunk EGK25-MB-M-B Greifer und einem Raspberry Pi 4 als Modbus-RTU-Gateway umgesetzt und schrittweise validiert. Ziel ist eine serielle, reproduzierbare Ansteuerung des Greifers inklusive Initialisierung, Statusauswertung und Bewegungsbefehlen. Die Verbindung wird im Testbetrieb stabil betrieben; Kommunikationsabläufe und Zustandsübergänge werden über Protokollausgaben, Registerwerte und Referenzvergleiche nachvollziehbar verifiziert.

Zu Beginn werden die grundlegenden Kommunikationsparameter festgelegt. Dazu gehören das serielle Gerät, Baudrate, Parität, Stopbits sowie die Slave-Adresse. Darauf aufbauend wird ein Testwerkzeug umgesetzt, das Modbus-RTU-Telegramme gemäß Spezifikation erzeugt und versendet. Ein zentraler Implementierungspunkt ist die CRC-16-Prüfsumme, da der Greifer ausschließlich Telegramme mit korrekt berechneter CRC verarbeitet und nur dann valide Antworttelegramme liefert. Erste Versuche mit einem Python-Skript dienen der Verifikation der Registerkommunikation. Dabei funktionieren Lesezugriffe stabil: Status- und Identifikationsregister lassen sich zuverlässig abfragen, und Antworttelegramme sind konsistent dekodierbar. Die

erfolgreiche Kommunikation auf Registerebene ist exemplarisch in der Terminalausgabe des Raspberry Pi dokumentiert.

Für die vollständige Steuerung (inklusive Write-Zugriffen und definierter Steuersequenzen) wird die Implementierung anschließend in Java umgesetzt. Das Java-Programm basiert auf einer eigenen Serialport-Integration und bildet die Modbus-RTU-Logik so ab, dass serielle Pufferung, Zeitabstände zwischen Telegrammen und Wiederholmechanismen gezielt kontrolliert werden können. Hierzu wird eine angepasste SerialPortWrapper-Schnittstelle implementiert, um die serielle Schicht stabil anzusprechen und Timing-Einflüsse reproduzierbar zu variieren. Die zyklische Verarbeitung der Telegramme sowie das zugehörige Cycle-Logging sind in der erweiterten Terminalausgabe des Raspberry Pi dargestellt (Abbildung 13)



```
rob@raspberrypi: ~/Seminararbeit
File Edit Tabs Help
rob@raspberrypi:~$ cd /home/rob/Seminararbeit
rob@raspberrypi:~/Seminararbeit$ java -jar build/libs/*.jar
Serial OK. Startup...
=== STARTUP BEGIN (FW5) ===
BOOT base 0 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 1 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 2 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 3 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 4 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 5 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 6 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 7 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 8 IN=0082 8000 003C 0000 0000 0000 00EF 0000
BOOT base 9 IN=0082 8000 003C 0000 0000 0000 00EF 0000
ACK pulse (0x00000005) ...
after ACK=1 IN=0023 8000 003C 0000 0000 0000 0000 0000
after ACK=0 IN=0023 8000 003C 0000 0000 0000 0000 0000
StatusDw=0x80000023 [ready:1 err:0]
=== STARTUP END (ready) ===
Alive loop running...
ABS MOVE -> pos=26500µm vel=40000µm/s ctrl=0x00002041 (cmdToggleBefore=1)
Command accepted: cmdToggle 1 -> 0 (Status=0x00000003)
Move done: Status=0x00002013
ABS MOVE -> pos=0µm vel=40000µm/s ctrl=0x00002001 (cmdToggleBefore=0)
```

Abbildung 13 Terminalausgabe Raspberry PI mit Zyklus-Log

Ein wesentlicher Validierungsschritt ist die Initialisierung des Greifers in einen betriebsbereiten Zustand. Nach dem Einschalten liegt ein Zustand vor, in dem Bewegungsbefehle abhängig von Register- und Statusbits blockiert werden können. Durch gezieltes Schreiben in die dafür vorgesehenen Register wird der Fehlerzustand quittiert und anschließend über Statusregister geprüft, ob der Greifer als bereit gemeldet wird. Diese Zustandsänderung wird zusätzlich durch Abgleich mit dem

Schunk Control Center verifiziert. Damit ist nachgewiesen, dass der Raspberry Pi mit dem Java-Programm gültige Telegramme erzeugt, die vom Greifer korrekt interpretiert und verarbeitet werden.

Aufbauend auf der erfolgreichen Initialisierung wird die Bewegungssteuerung umgesetzt. Dabei wird eine definierte Abfolge implementiert, die neben dem eigentlichen Bewegungsbefehl auch die notwendige Freigabe und Statusprüfung umfasst (Fehlerquittierung, Enable/Valid, Ready-Prüfung). In der praktischen Umsetzung zeigt sich, dass nicht nur die formale Korrektheit der Telegramme (Adressierung, Funktionscode, Register, CRC) relevant ist, sondern auch Reihenfolge und Zeitabstände zwischen den Telegrammen. Um diese Anforderungen reproduzierbar abzuleiten, werden Telegramme aufgezeichnet und mit Referenzabläufen aus dem Schunk Control Center sowie einem Telegramm-Analyzer verglichen. Die daraus abgeleitete Sequenz wird in Java als Ablauf implementiert und über Statusregister verifiziert. Nach Umsetzung der Sequenz und geeigneter Parametrierung der Timing-Abstände reagiert der Greifer zuverlässig auf Bewegungsbefehle; die Statusregister zeigen die erwarteten Zustandsübergänge, und die mechanische Bewegung erfolgt entsprechend der Sollvorgabe.

Zur Stabilisierung des Betriebs wird zusätzlich eine zyklische Abfrage (Keep-alive/Heartbeat) umgesetzt. Dabei werden in definierten Intervallen Statusregister gelesen, um den Gerätezustand kontinuierlich zu überwachen und Kommunikationsabbrüche frühzeitig zu erkennen. Die zyklische Protokollierung (Cycle-Logging) ermöglicht die Nachvollziehbarkeit von Telegrammfolgen, Zeitabständen und Zustandswechseln und dient als Grundlage für die systematische Anpassung von Wiederholungen und Wartezeiten. In wiederholten Testläufen werden Steuer- und Bewegungsbefehle stabil ausgeführt; die Kommunikation bleibt dabei konsistent, und Abweichungen werden über Statusbits eindeutig erkennbar.

Der Übergang vom initialen Zustand in den betriebsbereiten Zustand ist zusätzlich über die Status-LEDs am Greifer erkennbar. Abbildung 14 zeigt den Greifer unmittelbar nach dem Einschalten, Abbildung 15 den stabilen Betriebszustand nach erfolgreicher Initialisierung.



Abbildung 14 Status LED Greifer während Booten



Abbildung 15 Status LED Greifer nach Booten

Zusammenfassend wird die Anbindung des Schunk EGK25 an einen Raspberry Pi über Modbus RTU praktisch umgesetzt: Die serielle Verbindung ist stabil, der Greifer kann reproduzierbar initialisiert und freigegeben werden, Statusinformationen werden zyklisch ausgewertet, und Bewegungsbefehle werden zuverlässig übertragen und ausgeführt. Damit steht eine belastbare Grundlage zur Verfügung, um die Greifersteuerung strukturiert zu erweitern und in übergeordnete Abläufe zu integrieren.

8. Ausblick

Die Arbeit zeigt, dass die serielle Modbus-RTU-Kommunikation zwischen Raspberry Pi 4 und Schunk EGK25-MB-M-B zuverlässig umgesetzt werden kann. Auf dieser Basis sind Initialisierung, Statusauswertung und Bewegungsbefehle reproduzierbar ausführbar. Der Ausblick beschreibt Erweiterungen, die auf der funktionsfähigen Implementierung aufbauen und den Übergang von einer Test-/Laboransteuerung zu einer robusten Integrationslösung unterstützen.

Ein nächster Schritt besteht darin, die bestehende Java-Implementierung zu einem klar strukturierten „Command Layer“ auszubauen. Dabei werden wiederkehrende Abläufe (Initialisierung, Enable/Valid, Bewegungsbefehl, Abschlussprüfung, Fehlerquittierung) als definierte Befehlsketten kapsuliert. Ziel ist eine eindeutige Schnittstelle, die Greiferaktionen als Funktionen (z. B. Open, Close, MoveToPosition, SetForce, Stop) bereitstellt und die interne Registersequenz einschließlich Wartezeiten, Quittierungen und Zustandsprüfungen automatisch abwickelt. Dadurch wird die Ansteuerung konsistenter und weniger fehleranfällig.

Darauf aufbauend kann die Statusüberwachung weiter systematisiert werden. Dazu gehören ein einheitliches Zustandsmodell (z. B. Ready, Busy, Error, Homing/Init), zyklische Statusabfragen mit klaren Grenzwerten sowie eine saubere Fehlerklassifikation (kommunikationsbedingt vs. gerätebedingt). Ergänzend sind Wiederholmechanismen, Timeout-Überwachungen und definierte Recovery-Abläufe sinnvoll, um das Verhalten bei kurzzeitigen Störungen oder ungültigen Zuständen kontrolliert zu behandeln. Eine strukturierte Protokollierung (z. B. Zykluslogs, Ereignislogs, Fehlerlogs) erhöht dabei die Nachvollziehbarkeit und erleichtert die Diagnose.

Ein weiterer Ausbau betrifft die Parametrierung und Prozessfähigkeit der Greiferbewegungen. Auf Basis der stabilen Kommunikationsschicht können Parameter wie Greifkraft, Geschwindigkeit, Positionen und Zwischenpositionen als wiederverwendbare Profile abgelegt und gezielt aufgerufen werden. Dadurch lassen sich Laborversuche und Demonstrationsabläufe standardisieren, ohne die Registerebene manuell anpassen zu müssen.

Für eine spätere Integration in übergeordnete Systeme ist es sinnvoll, die Gateway-Funktion des Raspberry Pi auszubauen. Dazu kann die RTU-Anbindung intern beibehalten und nach außen über eine netzwerkbasierte Schnittstelle bereitgestellt werden (z. B. Modbus TCP oder eine schlanke API). Damit könnte ein Robotercontroller oder eine PC-Anwendung den Greifer über Ethernet ansprechen, während die serielle RTU-Kommunikation ausschließlich vom Gateway gehandhabt wird. Diese Trennung reduziert Komplexität im übergeordneten System und erleichtert die Wiederverwendung der Lösung in anderen Umgebungen.

Parallel dazu kann die Bedienbarkeit verbessert werden. Eine einfache textbasierte Oberfläche (CLI) oder eine kompakte GUI kann Zustände anzeigen, Befehle auslösen und Diagnosedaten strukturiert darstellen. Insbesondere eine übersichtliche Visualisierung relevanter Registerwerte und Statusbits unterstützt schnelle Fehlersuche und erleichtert die Nutzung im Laborbetrieb.

Zur Absicherung der Reproduzierbarkeit ist schließlich eine automatisierte Testumgebung sinnvoll. Diese kann definierte Sequenzen (inklusive Fehlerfällen) wiederholt ausführen, Timing-Varianten systematisch prüfen und Ergebnisse protokollieren. Auf diese Weise lassen sich Stabilität, Grenzfälle und Regressionen objektiv bewerten, bevor die Lösung in einen dauerhaften Demonstrations- oder Integrationsbetrieb überführt wird.

9. Fazit

Im Rahmen der Arbeit wird ein Vorgehen erarbeitet, das sich für Integrationsaufgaben an Systemen mit begrenzten Plattformressourcen als geeignet erweist. Es zeigt sich, dass die Realisierbarkeit weniger vom Protokollprinzip abhängt, sondern maßgeblich von den Randbedingungen der Zielplattform sowie von der korrekten Abfolge, dem Timing und der Zustandslogik der Kommunikations- und Steuersequenzen. Für die betrachtete CB3-Umgebung ist insbesondere die klare Trennung zwischen roboterseitiger Ablaufsteuerung und externer Kommunikationslogik entscheidend, um ein reproduzierbares Verhalten zu erreichen.

Weiterhin wird deutlich, dass eine stabile Ansteuerung nicht durch einzelne, formal korrekte Telegramme entsteht, sondern durch einen definierten und wiederholbaren Ablauf. Dazu gehören eine deterministische Initialisierung, eindeutige Freigabe- und Statusprüfungen, festgelegte Wiederhol- und Abbruchkriterien sowie eine strukturierte Protokollierung. Die systematische Umsetzung dieser Elemente ermöglicht eine nachvollziehbare Diagnose und eine kontrollierte Stabilisierung des Gesamtablaufs.

Die Gateway-Architektur unterstützt dieses Vorgehen, da Kommunikationslogik, Tests und Parametrierung unabhängig vom Robotercontroller durchgeführt werden können. Damit wird die Komplexität im Roboter reduziert und gleichzeitig eine Basis geschaffen, auf der Erweiterungen und Integrationsschritte konsistent aufgebaut werden können. Für den Laborbetrieb ist dies insbesondere relevant, weil Änderungen und Tests unter definierten Bedingungen erfolgen und Auswertungen über Logdaten objektiv durchgeführt werden können.

10. Quellen- und Literaturverzeichnis

[1] Universal Robots (2024). UR5 CB3 User Manual, Firmware Update Guide. Odense, Denmark: Universal Robots.

<https://www.universal-robots.com/support/manuals/>

[2] Schunk GmbH & Co. KG (2023). EGK25 Series User Manual. Lauffen am Neckar, Germany: Schunk.

<https://www.schunk.com/de/de/service/downloads>

[3] Modbus Organization (2023). Modbus Protocol Specification.

<https://www.modbus.org/specs.php>

[4] Paessler AG (2023). Modbus Explained: RTU vs TCP.

<https://www.paessler.com/it-explained/modbus>

[5] Raspberry Pi Foundation (2024). *Raspberry Pi hardware documentation*.

<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>

[6] Oracle & Python Software Foundation (2024). Java Platform Overview; Python Documentation Overview.

<https://docs.oracle.com/en/java/>

<https://docs.python.org/3/>

11. Anhang

Hinweis: als Text formatiert (monospace) und entsprechen dem im Projekt verwendeten Stand.

A: ExampleInstallationNode.java

```
package com.example.urcap;
import com.ur.urcap.api.domain.installation.InstallationAPI;
import com.ur.urcap.api.domain.installation.InstallationNodeContribution;
import com.ur.urcap.api.domain.program.ScriptWriter;
//Getestet unter Ubuntu 22.04 mit URCap SDK 1.15 und Java 17.
//Der Node zeigt beim Starten des Programms nur ein einfaches Popup am Roboter.
public class ExampleInstallationNode implements InstallationNodeContribution {
    private final InstallationAPI api;
    private final ExampleInstallationNodeView view;
    public ExampleInstallationNode(InstallationAPI api, ExampleInstallationNodeView view) {
        this.api = api;
        this.view = view;
    }
    @Override
    public void openView() {
        System.out.println("[URCap] Installation Node geöffnet – View wird geladen...");
        view.buildUI();
    }
    @Override
    public void closeView() {
        // Wird beim Verlassen des Nodes ausgeführt.
        System.out.println("[URCap] Installation Node geschlossen.");
    }
    @Override
    public void generateScript(ScriptWriter writer) {
        // Hier werden die Befehle definiert
        // Im Beispiel nur ein einfaches Popup
        writer.appendLine("popup(\"Hello from URCap Example on Ubuntu!\")");
    }
}
```

B: ExampleInstallationNodeView.java

```
package com.example.urcap;
import com.ur.urcap.api.domain.userinteraction.swing.SwingUserInterface;
import javax.swing.*;
//Diese Klasse erzeugt die grafische Oberfläche für den Installations-Node.
//Alles, was der Benutzer im Teach Pendant sieht, wird hier aufgebaut.
public class ExampleInstallationNodeView {
    private JPanel mainPanel;
    public ExampleInstallationNodeView() {
        // Grundpanel initialisieren
        mainPanel = new JPanel();
    }
    public void buildUI() {
        //Beim Öffnen des Nodes wird die UI jedes Mal neu aufgebaut
        mainPanel.removeAll();
        JLabel label = new JLabel("URCap Installation Node Example (Ubuntu Test)");
        JButton button = new JButton("Say Hello");
        //Dialoge werden im Teach Pendant sichtbar angezeigt
        button.addActionListener(e -> JOptionPane.showMessageDialog(
            null,
            "Button gedrückt – funktioniert auch direkt auf dem Pendant!",
            "URCap Test",
            JOptionPane.INFORMATION_MESSAGE
        ));
        mainPanel.add(label);
        mainPanel.add(button);
    }
}
```

```
        mainPanel.revalidate();
        mainPanel.repaint();
        System.out.println("[URCap] UI aufgebaut und Komponenten geladen.");
    }
    public SwingUserInterface getUI() {
        // Liefert das Hauptpanel an das URCap-Framework zurück.
        //ist Wichtig, sonst wird nichts angezeigt.
        return new SwingUserInterface(mainPanel);
    }
}
```

C: build.gradle

```
plugins {
    id 'java'
}
group 'com.example'
version '1.0-SNAPSHOT'

sourceCompatibility = '17'
targetCompatibility = '17'

repositories {
    mavenCentral()
}
dependencies {
    // Pfad muss auf die SDK-Libs deines URCap-Installationsverzeichnis zeigen.
    // Bei Ubuntu liegt das normalerweise unter ~/urcaps/sdk/lib/
    implementation fileTree(dir: '/home/rob/urcaps/sdk/lib', include: '*.jar')
}

jar {
    archiveBaseName.set('ExampleURCap')
    archiveVersion.set('1.0')
    manifest {
        attributes 'Main-Class': 'com.example.urcap.ExampleInstallationNode'
    }
}
```

D: SchunkGripperControl.java

```
public class SchunkGripperControl {

    // ===== Konfiguration =====
    private static final String PORT_NAME = "/dev/ttyACM0"; // Serieller Port des Greifers
    private static final int SLAVE_ID = 10; // Modbus ID des Greifers

    // Register-Adressen
    private static final int START_REGISTER = 0x0047;
    private static final int STATUS_REGISTER = 0x003F;
    private static final int POSITION_REGISTER = 0x0040;

    // Steuerbefehle
    private static final int INIT_SIGNAL = 0x0064;
    private static final int ACK_SIGNAL = 0x0004;
    private static final int ERROR_ACK = 0x0024;
    private static final int ENABLE_VALID = 0x0180;
    private static final int GENERAL_RESET = 0x0001;
    private static final int HARDWARE_RESET = 0x0080;

    public static void main(String[] args) throws InterruptedException {
        // Seriellen Port konfigurieren
        SerialPort port = SerialPort.getCommPort(PORT_NAME);
        port.setBaudRate(19200);
        port.setNumDataBits(8);
        port.setNumStopBits(1);
    }
}
```

```
port.setParity(SerialPort.EVEN_PARITY);  
port.setComPortTimeouts(SerialPort.TIMEOUT_READ_BLOCKING, 1000, 0);
```

E: Java-Beispielprogramm Modbus-RTU-Steuerung

```
import com.fazecast.jSerialComm.SerialPort;  
/**  
 * Java-Beispielprogramm zur Steuerung des Schunk EGK25-MB-M-B Greifers  
 * über Modbus RTU mit einem Raspberry Pi.  
 * Das Programm demonstriert:  
 * - Aufbau einer stabilen seriellen Verbindung  
 * - Initialisierung des Greifers  
 * - Quittierung von Fehlerzuständen  
 * - Aktivierung des Greifers (Enable + Valid)  
 * - Auslesen des Statusregisters und Prüfung der Betriebsbereitschaft  
 * - Modularer Aufbau zur einfachen Erweiterung (Heartbeat, Bewegungsbefehle)  
 */  
public class SchunkGripperControl {  
  
    // Konfiguration  
    // =====  
    private static final String PORT_NAME = "/dev/ttyACM0"; // Serieller Port des Greifers  
    private static final int SLAVE_ID = 10;                // Modbus Slave-ID  
  
    // Register-Adressen laut Hersteller-Dokumentation  
    private static final int START_REGISTER = 0x0047;      // Steuerregister  
    private static final int STATUS_REGISTER = 0x003F;     // Statusregister  
    private static final int POSITION_REGISTER = 0x0040;    // Positionsregister (optional)  
  
    // Steuerbefehle laut Hersteller  
    private static final int INIT_SIGNAL = 0x0064;         // Initialisierung  
    private static final int ACK_SIGNAL = 0x0004;          // Fehlerquittierung  
    private static final int ENABLE_VALID = 0x0180;        // Greifer aktivieren  
    private static final int GENERAL_RESET = 0x0001;       // Allgemeiner Reset  
    private static final int HARDWARE_RESET = 0x0080;      // Hardware Reset  
  
    public static void main(String[] args) throws InterruptedException {  
        SerialPort port = SerialPort.getCommPort(PORT_NAME);  
        configureSerialPort(port);  
    }  
}
```